

Object Oriented Programming (OOP)

Semester: 2nd

Credit Hour: (01)

Department of Computer Science

Course Instructor: Engr. Zakria Bacha

Lab Task No. 10

Structure in C++

Previous Lab:

In previous lab, we have covered below three topics:

1. Destructor in C++
2. Execution order of destructor in C++
3. Static data members in C++
4. Static methods in c++
5. Dynamic Memory management Using Destructor

Current Topics for Lab:

1. Operator Overloading
2. Shallow Copy Vs Deep Copy constructor in C++

What is Operator overloading in C++:

Operator overloading is a feature in programming that lets you **redefine how standard operators** (like +, -, *, ==, etc.) behave when applied to user-defined types (like classes or structs).

OR

When an operator is overloaded with multiple jobs, it is known as operator overloading.

Syntax:

```
return_type operator <symbol>(parameters)
{
    // implementation
}
```

Figure 1

Why To Use Operator Overloading:

Custom Types (User defined data types) work Like Built-in Types:

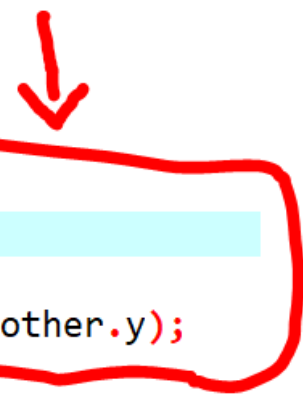
Your Complex, Matrix, or Vector class behaves such as like int or float:

```
Complex c1(2,3), c2(1,4);
Complex c3 = c1 + c2; // similar to just like int addition
cout << c3; // similar to just like printing an int
```

Adding Two objects in Dev C++:

Class Code:

```
4 class Vector {  
5  
6     private:  
7         int x, y;  
8     public:  
9         Vector(int x, int y) : x(x), y(y)  
10        {  
11  
12        }  
13        // Overload the + operator  
14        Vector operator+(Vector& other)  
15        {  
16            return Vector(x + other.x, y + other.y);  
17        }  
18  
19        void print()  
20        {  
21            cout<<x<<" "<<y<<endl;  
22        }  
23    };
```



Main Function Code:

```
25 int main()  
26 {  
27     Vector v1(1, 2), v2(3, 4);  
28  
29     Vector v3 = v1 + v2; // calls operator+  
30  
31     v3.print(); // Output: (4, 6)  
32 }
```

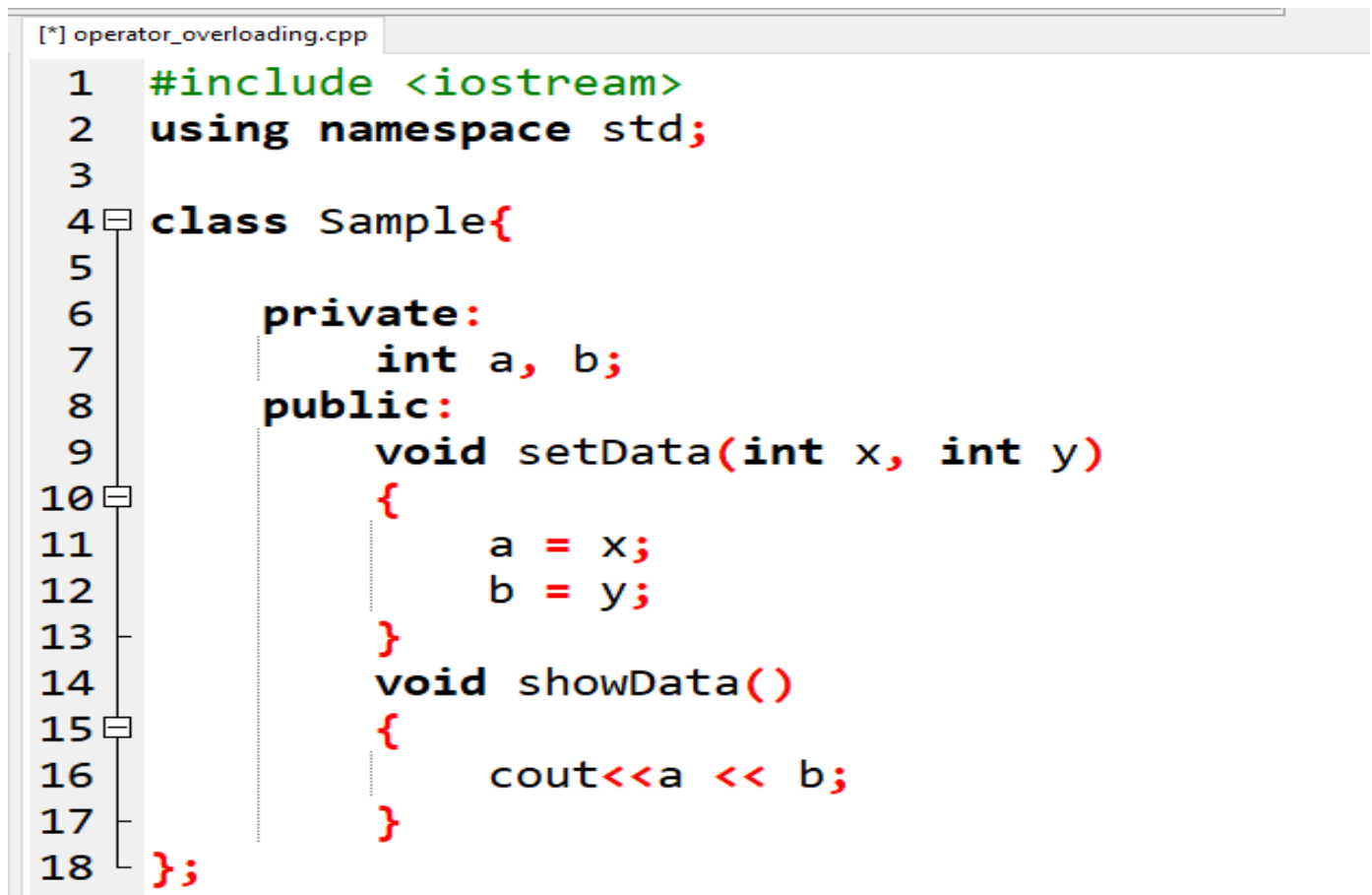
Important Points regarding Operator Overloading:

1. You must use Operator while doing operator overloading.
2. You can not change precedence like * has greater precedence than + operator.
3. You can not overload **. :: ?: sizeof.**
4. At least one user-defined type

Deep copy & Shallow copy in C++?

Copy.

How can we create a copy of object?

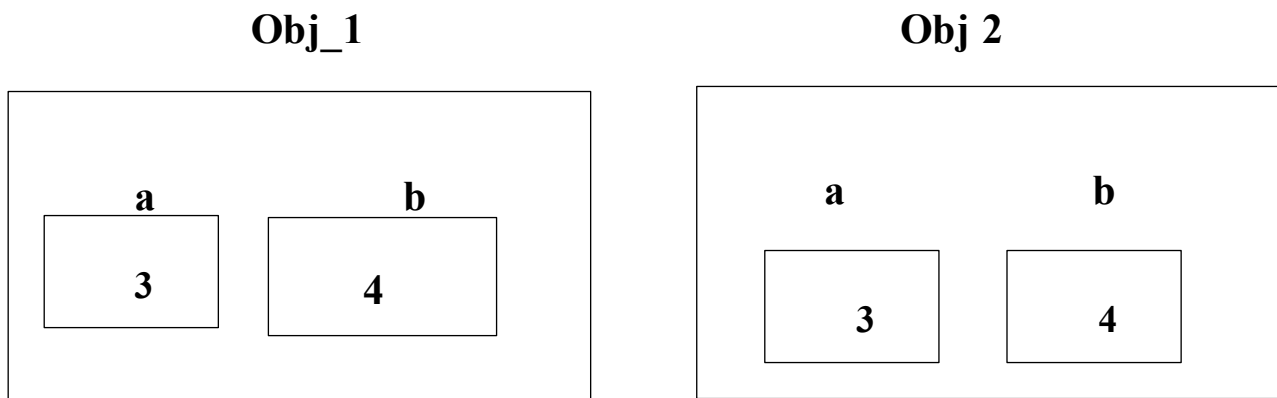


```
[*] operator_overloading.cpp
1  #include <iostream>
2  using namespace std;
3
4  class Sample{
5
6      private:
7          int a, b;
8      public:
9          void setData(int x, int y)
10         {
11             a = x;
12             b = y;
13         }
14         void showData()
15         {
16             cout<<a << b;
17         }
18     };
```

Figure 2

```
20 int main()  
21 {  
22     Sample obj_1;  
23     obj_1.setData(3,4);  
24  
25     Sample obj_2 = obj_1; // obj_1 data is copied  
26  
27     obj_2.showData();  
28
```

Figure 3



Data is copied in two ways:

- Copy Constructor (made by compiler)
- Implicit Copy assignment Operator (overload by constructor)

```
20 int main()
21 {
22     Sample obj_1;
23     obj_1.setData(3,4);
24
25     Sample obj_2;
26
27     obj_2 = obj_1; // obj_1 data is copied
28
29     obj_2.showData();
30 }
```

Difference between Shallow copy Vs Deep Copy:

Shallow Copy:

Creating copy of object by coping data of all member variable as it is.

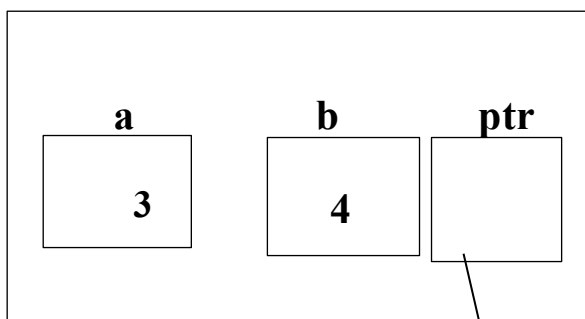
```
Sample(Sample &s) // Shallow copy
{
    a = s.a;
    b = s.b;
}
```

Deep Copy:

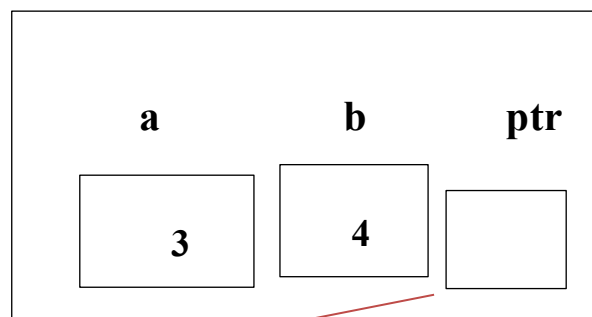
Creating an object by coping data of another object along with the values of memory resources resides outside the object but handled by that object.

```
[*] operator_overloading.cpp
4 class Sample{
5
6     private:
7         int a, b, *ptr;
8     public:
9
10        Sample()
11        {
12            a = 0;
13            b = 0;
14            ptr = new int;
15        }
16        void setData(int x, int y, int z)
17        {
18            a = x;
19            b = y;
20            *ptr = z;
21        }
```

Obj_1



Obj 2



5

```
26  
27 Sample(Sample &s) // shallow copy  
28 {  
29     a = s.a;  
30     b = s.b;  
31     ptr = s.ptr;  
32 }  
33 };
```

You should make copy constructor for deep copy.

You should not copy the data of member variable but the value to which member variable pointer to.


```
operator_overloading.cpp
26     Sample(Sample &s) // Deep copy
27     {
28         a = s.a;
29         b = s.b;
30         ptr = new int;
31         *ptr = *(s.ptr);
32     }
33 };
34
35 int main()
36 {
37     Sample obj_1;
38     obj_1.setData(3,4,5);
39
40     Sample obj_2;
41
42     obj_2 = obj_1; // obj_1 data is copied
43
44     obj_2.showData();
45
46 }
```

Figure 4

To avoid Dangling Pointer

Figure 5

Create a class `Distance` that stores distance in feet and inches as private members. Overload the `+` operator to add two `Distance` objects together (handling inch-to-feet conversion) and overload the `==` operator to compare whether two distances are equal. Provide a `setData()` function to assign values and a `showData()` function to display results, then in `main()` create two `Distance` objects, add them using `+`, compare them using `==`, and print all results to the screen.

Ahmed is building a student record system using a Student class that stores name dynamically with char* name pointer and a roll number. Help Ahmed by allocating memory in constructor using new, freeing it in destructor using delete[], creating object obj_1 for student Usman with roll number 101, copying it into obj_2 using default compiler copy, and printing both objects using showData().

Ahmed wants to fix his student record system from Task 2 by writing his own copy constructor. Help Ahmed by implementing a deep copy constructor in the same Student class that allocates new separate memory using new and copies the actual name value using strcpy,

then in main create obj_1 for Usman and copy it into obj_2 and print both objects using showData() to verify both records are independent.

National University of Computer and Emerging Sciences

Peshawar Campus